

AI-powered FAQ and customer support automation

Complete Project Source Code

Project	AI-powered FAQ and customer support automation
Repository	https://github.com/thushetha30052007-maker/AI-powered-FAQ-and-customer-support-automation/tree/main/CODE
Document Type	Single PDF source-code listing
Security Note	Live environment secrets are intentionally redacted.

Project Structure

```
src/
|-- config/
|   |-- db.js
|-- controllers/
|   |-- aiController.js
|   |-- authController.js
|   |-- faqController.js
|-- middleware/
|   |-- authMiddleware.js
|   |-- errorMiddleware.js
|   |-- validationMiddleware.js
|-- models/
|   |-- FAQ.js
|   |-- User.js
|-- routes/
|   |-- aiRoutes.js
|   |-- authRoutes.js
|   |-- faqRoutes.js
|-- services/
|   |-- geminiService.js
|-- utils/
|   |-- helpers.js
|-- app.js
|-- server.js
.env
.gitignore
package-lock.json
package.json
README.md
```

Scope: This PDF lists the application source files and key runtime configuration. **package-lock.json** is an auto-generated dependency lockfile rather than application source code and is not reproduced. **.gitignore** is visible in the repository tree, but its exact contents are not present in the local source archive used for this document, so they are not fabricated.

File Index

1. **src/config/db.js**
2. **src/controllers/aiController.js**
3. **src/controllers/authController.js**
4. **src/controllers/faqController.js**
5. **src/middleware/authMiddleware.js**
6. **src/middleware/errorMiddleware.js**
7. **src/middleware/validationMiddleware.js**
8. **src/models/FAQ.js**
9. **src/models/User.js**
10. **src/routes/aiRoutes.js**
11. **src/routes/authRoutes.js**
12. **src/routes/faqRoutes.js**
13. **src/services/geminiService.js**
14. **src/utils/helpers.js**
15. **src/app.js**
16. **src/server.js**
17. **.env**
18. **package.json**
19. **README.md**

src/config/db.js

```
1 | const mongoose = require('mongoose');
2 |
3 | const connectDB = async () => {
4 |   const uri = process.env.MONGO_URI;
5 |   if (!uri) {
6 |     throw new Error('MONGO_URI is not configured');
7 |   }
8 |
9 |   const conn = await mongoose.connect(uri);
10 |   console.log(`MongoDB connected: ${conn.connection.host}`);
11 |   return conn;
12 | };
13 |
14 | module.exports = connectDB;
```

src/controllers/aiController.js

```
1 | const { generateAnswer, generateFAQ } = require('../services/geminiService');
2 | const { asyncHandler } = require('../utils/helpers');
3 |
4 | const generateAIAnswer = asyncHandler(async (req, res) => {
5 |   const answer = await generateAnswer(req.body.question);
6 |   res.json({ success: true, data: { question: req.body.question, answer } });
7 | });
8 |
9 | const generateAIFAQ = asyncHandler(async (req, res) => {
10 |   const faq = await generateFAQ(req.body.topic);
11 |   res.json({ success: true, data: { topic: req.body.topic, ...faq } });
12 | });
13 |
14 | module.exports = { generateAIAnswer, generateAIFAQ };
```

src/controllers/authController.js

```
1 | const User = require('../models/User');
2 | const { generateToken, asyncHandler } = require('../utils/helpers');
3 |
4 | const registerUser = asyncHandler(async (req, res) => {
5 |   const { name, email, password } = req.body;
6 |   const exists = await User.findOne({ email: email.toLowerCase() });
7 |   if (exists) return res.status(400).json({ success: false, message: 'User already exists' });
8 |
9 |   const user = await User.create({ name, email, password });
10 |   res.status(201).json({
11 |     success: true,
12 |     data: { id: user._id, name: user.name, email: user.email, role: user.role, token: generateToken(user._id) },
13 |   });
14 | });
15 |
16 | const loginUser = asyncHandler(async (req, res) => {
17 |   const { email, password } = req.body;
18 |   const user = await User.findOne({ email: email.toLowerCase() }).select('+password');
19 |   if (!user || !(await user.matchPassword(password))) {
20 |     return res.status(401).json({ success: false, message: 'Invalid email or password' });
21 |   }
22 |
23 |   res.json({
24 |     success: true,
25 |     data: { id: user._id, name: user.name, email: user.email, role: user.role, token: generateToken(user._id) },
26 |   });
27 | });
28 |
29 | const getUserProfile = asyncHandler(async (req, res) => {
30 |   res.json({ success: true, data: req.user });
31 | });
32 |
33 | module.exports = { registerUser, loginUser, getUserProfile };
```

src/controllers/faqController.js

```
1 | const FAQ = require('../models/FAQ');
2 | const { asyncHandler } = require('../utils/helpers');
3 |
4 | const getAllFAQs = asyncHandler(async (req, res) => {
5 |   const faqs = await FAQ.find().populate('createdBy', 'name email').sort({ createdAt: -1 });
6 |   res.json({ success: true, count: faqs.length, data: faqs });
7 | });
8 |
9 | const getFAQById = asyncHandler(async (req, res) => {
10 |   const faq = await FAQ.findById(req.params.id).populate('createdBy', 'name email');
11 |   if (!faq) return res.status(404).json({ success: false, message: 'FAQ not found' });
12 |   res.json({ success: true, data: faq });
13 | });
14 |
15 | const createFAQ = asyncHandler(async (req, res) => {
16 |   const faq = await FAQ.create({ ...req.body, createdBy: req.user._id });
17 |   res.status(201).json({ success: true, data: faq });
18 | });
19 |
20 | const updateFAQ = asyncHandler(async (req, res) => {
21 |   const faq = await FAQ.findById(req.params.id);
22 |   if (!faq) return res.status(404).json({ success: false, message: 'FAQ not found' });
23 |
24 |   const isOwner = faq.createdBy.toString() === req.user._id.toString();
25 |   const isAdmin = req.user.role === 'admin';
26 |   if (!isOwner && !isAdmin) return res.status(403).json({ success: false, message: 'Not authorized to update this FAQ' });
27 |
28 |   const allowed = ['question', 'answer', 'category'];
29 |   for (const key of allowed) if (req.body[key] !== undefined) faq[key] = req.body[key];
30 |   await faq.save();
31 |   res.json({ success: true, data: faq });
32 | });
33 |
34 | const deleteFAQ = asyncHandler(async (req, res) => {
35 |   const faq = await FAQ.findById(req.params.id);
36 |   if (!faq) return res.status(404).json({ success: false, message: 'FAQ not found' });
37 |
38 |   const isOwner = faq.createdBy.toString() === req.user._id.toString();
39 |   const isAdmin = req.user.role === 'admin';
40 |   if (!isOwner && !isAdmin) return res.status(403).json({ success: false, message: 'Not authorized to delete this FAQ' });
41 |
42 |   await faq.deleteOne();
43 |   res.json({ success: true, message: 'FAQ deleted' });
44 | });
45 |
46 | const searchFAQs = asyncHandler(async (req, res) => {
47 |   const q = String(req.query.q || '').trim();
48 |   if (!q) return res.status(400).json({ success: false, message: 'Search query q is required' });
49 |
50 |   const regex = new RegExp(q.replace(/[*?^$(){}|\[\]\\\]/g, '\\$&'), 'i');
51 |   const faqs = await FAQ.find({
52 |     $or: [{ question: regex }, { answer: regex }, { category: regex }],
53 |   }).sort({ createdAt: -1 });
54 |
55 |   res.json({ success: true, count: faqs.length, query: q, data: faqs });
56 | });
57 |
```

```
58 | module.exports = { getAllFAQs, getFAQById, createFAQ, updateFAQ, deleteFAQ, searchFAQs };
```

src/middleware/authMiddleware.js

```
1 | const jwt = require('jsonwebtoken');
2 | const User = require('../models/User');
3 |
4 | const protect = async (req, res, next) => {
5 |   try {
6 |     const auth = req.headers.authorization || '';
7 |     if (!auth.startsWith('Bearer ')) {
8 |       return res.status(401).json({ success: false, message: 'Not authorized, no token' });
9 |     }
10 |
11 |     const token = auth.split(' ')[1];
12 |     const decoded = jwt.verify(token, process.env.JWT_SECRET);
13 |     const user = await User.findById(decoded.id);
14 |
15 |     if (!user) {
16 |       return res.status(401).json({ success: false, message: 'Not authorized, user not found' });
17 |     }
18 |
19 |     req.user = user;
20 |     next();
21 |   } catch (error) {
22 |     return res.status(401).json({ success: false, message: 'Not authorized, token failed' });
23 |   }
24 | };
25 |
26 | module.exports = { protect };
```

src/middleware/errorMiddleware.js

```
1 | const errorHandler = (err, req, res, next) => {
2 |   let statusCode = err.status || (res.statusCode === 200 ? 500 : res.statusCode);
3 |   let message = err.message || 'Server error';
4 |
5 |   if (err.code === 11000) {
6 |     statusCode = 400;
7 |     const field = Object.keys(err.keyValue || {})[0] || 'field';
8 |     message = `Duplicate field value entered: ${field}. Please use another value.`;
9 |   }
10 |
11 |   if (err.name === 'ValidationError') {
12 |     statusCode = 400;
13 |     message = Object.values(err.errors).map((val) => val.message).join(', ');
14 |   }
15 |
16 |   if (err.name === 'CastError') {
17 |     statusCode = 404;
18 |     message = `Resource not found with id of ${err.value}`;
19 |   }
20 |
21 |   if (statusCode >= 500) console.error(err.stack || err);
22 |
23 |   res.status(statusCode).json({
24 |     success: false,
25 |     message,
26 |     ...(process.env.NODE_ENV === 'development' ? { stack: err.stack } : {}),
27 |   });
28 | };
29 |
30 | module.exports = { errorHandler };
```

src/middleware/validationMiddleware.js

```
1 | const fail = (res, message) => res.status(400).json({ success: false, message });
2 |
3 | const validateRegister = (req, res, next) => {
4 |   const { name, email, password } = req.body;
5 |   if (!name || !email || !password) return fail(res, 'name, email and password are required');
6 |   if (password.length < 6) return fail(res, 'Password must be at least 6 characters long');
7 |   next();
8 | };
9 |
10 | const validateLogin = (req, res, next) => {
11 |   const { email, password } = req.body;
12 |   if (!email || !password) return fail(res, 'email and password are required');
13 |   next();
14 | };
15 |
16 | const validateFAQ = (req, res, next) => {
17 |   const { question, answer, category } = req.body;
18 |   if (!question || !answer || !category) return fail(res, 'question, answer and category are required');
19 |   next();
20 | };
21 |
22 | const validateAIAnswer = (req, res, next) => {
23 |   const { question } = req.body;
24 |   if (!question || typeof question !== 'string') return fail(res, 'question is required');
25 |   next();
26 | };
27 |
28 | const validateAIFAQ = (req, res, next) => {
29 |   const { topic } = req.body;
30 |   if (!topic || typeof topic !== 'string') return fail(res, 'topic is required');
31 |   next();
32 | };
33 |
34 | module.exports = {
35 |   validateRegister,
36 |   validateLogin,
37 |   validateFAQ,
38 |   validateAIAnswer,
39 |   validateAIFAQ,
40 | };
```

src/models/FAQ.js

```
1 | const mongoose = require('mongoose');
2 |
3 | const faqSchema = new mongoose.Schema(
4 |   {
5 |     question: {
6 |       type: String,
7 |       required: [true, 'Question is required'],
8 |       trim: true,
9 |       index: 'text',
10 |     },
11 |     answer: {
12 |       type: String,
13 |       required: [true, 'Answer is required'],
14 |       trim: true,
15 |       index: 'text',
16 |     },
17 |     category: {
18 |       type: String,
19 |       required: [true, 'Category is required'],
20 |       enum: ['Technology', 'Education', 'Health', 'Banking', 'General'],
21 |       default: 'General',
22 |       index: true,
23 |     },
24 |     createdBy: {
25 |       type: mongoose.Schema.Types.ObjectId,
26 |       ref: 'User',
27 |       required: [true, 'Creator user reference is required'],
28 |       index: true,
29 |     },
30 |   },
31 |   { timestamps: true }
32 | );
33 |
34 | faqSchema.index({ question: 'text', answer: 'text', category: 'text' });
35 |
36 | module.exports = mongoose.model('FAQ', faqSchema);
```

src/models/User.js

```
1 | const mongoose = require('mongoose');
2 | const bcrypt = require('bcrypt');
3 |
4 | const userSchema = new mongoose.Schema(
5 |   {
6 |     name: {
7 |       type: String,
8 |       required: [true, 'Name is required'],
9 |       trim: true,
10 |    },
11 |    email: {
12 |      type: String,
13 |      required: [true, 'Email is required'],
14 |      unique: true,
15 |      lowercase: true,
16 |      trim: true,
17 |      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
18 |    },
19 |    password: {
20 |      type: String,
21 |      required: [true, 'Password is required'],
22 |      minlength: [6, 'Password must be at least 6 characters long'],
23 |      select: false,
24 |    },
25 |    role: {
26 |      type: String,
27 |      enum: ['user', 'creator', 'admin'],
28 |      default: 'user',
29 |    },
30 |  },
31 |   { timestamps: true }
32 | );
33 |
34 | userSchema.pre('save', async function () {
35 |   if (!this.isModified('password')) return;
36 |   const salt = await bcrypt.genSalt(10);
37 |   this.password = await bcrypt.hash(this.password, salt);
38 | });
39 |
40 | userSchema.methods.matchPassword = function (enteredPassword) {
41 |   return bcrypt.compare(enteredPassword, this.password);
42 | };
43 |
44 | module.exports = mongoose.model('User', userSchema);
```

src/routes/aiRoutes.js

```
1 | const express = require('express');
2 | const router = express.Router();
3 | const {
4 |   generateAIAnswer,
5 |   generateAIFAQ,
6 | } = require('../controllers/aiController');
7 | const { protect } = require('../middleware/authMiddleware');
8 | const {
9 |   validateAIAnswer,
10 |  validateAIFAQ,
11 | } = require('../middleware/validationMiddleware');
12 |
13 | // Protected routes to prevent API overuse/abuse
14 | router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 | router.post('/generate-faq', protect, validateAIFAQ, generateAIFAQ);
16 |
17 | module.exports = router;
```

src/routes/authRoutes.js

```
1 | const express = require('express');
2 | const router = express.Router();
3 | const {
4 |   registerUser,
5 |   loginUser,
6 |   getUserProfile,
7 | } = require('../controllers/authController');
8 | const { protect } = require('../middleware/authMiddleware');
9 | const {
10 |   validateRegister,
11 |   validateLogin,
12 | } = require('../middleware/validationMiddleware');
13 |
14 | // Public routes
15 | router.post('/register', validateRegister, registerUser);
16 | router.post('/login', validateLogin, loginUser);
17 |
18 | // Protected routes
19 | router.get('/profile', protect, getUserProfile);
20 |
21 | module.exports = router;
```

src/routes/faqRoutes.js

```
1 | const express = require('express');
2 | const router = express.Router();
3 | const {
4 |   getAllFAQs,
5 |   getFAQById,
6 |   createFAQ,
7 |   updateFAQ,
8 |   deleteFAQ,
9 |   searchFAQs,
10 | } = require('../controllers/faqController');
11 | const { protect } = require('../middleware/authMiddleware');
12 | const { validateFAQ } = require('../middleware/validationMiddleware');
13 |
14 | // Public routes - GET operations
15 | router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16 | router.get('/', getAllFAQs);
17 | router.get('/:id', getFAQById);
18 |
19 | // Protected routes - Modification operations
20 | router.post('/', protect, validateFAQ, createFAQ);
21 | router.put('/:id', protect, updateFAQ);
22 | router.delete('/:id', protect, deleteFAQ);
23 |
24 | module.exports = router;
```

src/services/geminiService.js

```
1 | const { GoogleGenAI } = require('@google/genai');
2 |
3 | const getClient = () => {
4 |   if (!process.env.GEMINI_API_KEY) {
5 |     const err = new Error('GEMINI_API_KEY is not configured');
6 |     err.status = 500;
7 |     throw err;
8 |   }
9 |   return new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
10 | };
11 |
12 | const generateText = async (prompt) => {
13 |   const ai = getClient();
14 |   const response = await ai.models.generateContent({
15 |     model: process.env.GEMINI_MODEL || 'gemini-2.5-flash',
16 |     contents: prompt,
17 |   });
18 |   return response.text;
19 | };
20 |
21 | const generateAnswer = (question) =>
22 |   generateText(`Answer the following customer-support question clearly and accurately. Keep the answer concise and practical.\n\nQuestion: ${question}`);
23 |
24 | const generateFAQ = async (topic) => {
25 |   const text = await generateText(
26 |     `Create one FAQ entry about the topic below. Return ONLY valid JSON with exactly these keys: question, answer, category. Category must be one of Technology, Education, Health, Banking, General.\n\nTopic: ${topic}`
27 |   );
28 |
29 |   const cleaned = text.replace(/^```json$/i, '').replace(/^```$/i, '').replace(/```$/i, '').trim();
30 |   try {
31 |     return JSON.parse(cleaned);
32 |   } catch {
33 |     return { question: `What should I know about ${topic}?`, answer: text.trim(), category: 'General' };
34 |   }
35 | };
36 |
37 | module.exports = { generateAnswer, generateFAQ };
```

src/utils/helpers.js

```
1 | const jwt = require('jsonwebtoken');
2 |
3 | const generateToken = (id) => {
4 |   const secret = process.env.JWT_SECRET;
5 |   if (!secret) throw new Error('JWT_SECRET is not configured');
6 |   return jwt.sign({ id }, secret, {
7 |     expiresIn: process.env.JWT_EXPIRES_IN || '7d',
8 |   });
9 | };
10 |
11 | const asyncHandler = (fn) => (req, res, next) => {
12 |   Promise.resolve(fn(req, res, next)).catch(next);
13 | };
14 |
15 | module.exports = { generateToken, asyncHandler };
```

src/app.js

```
1 | const express = require('express');
2 | const cors = require('cors');
3 | const dotenv = require('dotenv');
4 | const { errorHandler } = require('./middleware/errorMiddleware');
5 |
6 | dotenv.config();
7 | const app = express();
8 |
9 | app.use(cors());
10 | app.use(express.json());
11 | app.use(express.urlencoded({ extended: false }));
12 |
13 | app.get('/', (req, res) => {
14 |   res.json({ success: true, message: 'AI FAQ Assistant API is running.', version: '1.0.0' });
15 | });
16 |
17 | app.use('/api/auth', require('./routes/authRoutes'));
18 | app.use('/api/faqs', require('./routes/faqRoutes'));
19 | app.use('/api/ai', require('./routes/aiRoutes'));
20 |
21 | app.use((req, res) => res.status(404).json({ success: false, message: 'Route not found' }));
22 | app.use(errorHandler);
23 |
24 | module.exports = app;
```

src/server.js

```
1 | const app = require('./app');
2 | const connectDB = require('./config/db');
3 |
4 | const startServer = async () => {
5 |   await connectDB();
6 |
7 |   const PORT = process.env.PORT || 5000;
8 |   const server = app.listen(PORT, () => {
9 |     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
10 |   });
11 |
12 |   process.on('unhandledRejection', (err) => {
13 |     console.error(`Unhandled Rejection: ${err.message}`);
14 |     server.close(() => process.exit(1));
15 |   });
16 | };
17 |
18 | startServer().catch((err) => {
19 |   console.error('Failed to start server:', err);
20 |   process.exit(1);
21 | });
```

.env

Sanitized environment configuration. Secret values are deliberately redacted.

```
1 | PORT=5000
2 | NODE_ENV=development
3 | MONGO_URI=mongodb://127.0.0.1:27017/ai_faq_assistant
4 | JWT_SECRET=<REDACTED - keep your actual secret only in local .env>
5 | JWT_EXPIRES_IN=7d
6 | GEMINI_API_KEY=<REDACTED - keep your actual API key only in local .env>
7 | GEMINI_MODEL=gemini-2.5-flash
```

package.json

```
1 | {
2 |   "name": "ai-faq-assistant-api",
3 |   "version": "1.0.0",
4 |   "description": "Reconstructed runnable backend based on the uploaded AI FAQ Assistant document",
5 |   "main": "src/server.js",
6 |   "scripts": {
7 |     "start": "node src/server.js",
8 |     "dev": "nodemon src/server.js"
9 |   },
10 |   "dependencies": {
11 |     "@google/genai": "^1.0.0",
12 |     "bcrypt": "^6.0.0",
13 |     "cors": "^2.8.5",
14 |     "dotenv": "^17.0.0",
15 |     "express": "^5.1.0",
16 |     "jsonwebtoken": "^9.0.2",
17 |     "mongoose": "^8.0.0"
18 |   },
19 |   "devDependencies": {
20 |     "nodemon": "^3.1.0"
21 |   }
22 | }
```

README.md

Project documentation file included for completeness.

```
1 | # AI FAQ Assistant API - reconstructed runnable project
2 |
3 | This folder is a **reconstruction**, not a verbatim extraction of every file. The uploaded PDF shows the project architecture and screenshots for the gateway, middleware,
  | models, and routes, but it only *lists* controller/service/config/helper files without showing their code. The missing implementations here were created to match the
  | documented endpoints and behavior.
4 |
5 | ## Setup
6 | 1. Copy `.env.example` to `.env`.
7 | 2. Configure MongoDB, JWT secret, and Gemini API key.
8 | 3. Run `npm install`.
9 | 4. Run `npm run dev`.
10 |
11 | ## Documented endpoints
12 | - `POST /api/auth/register`
13 | - `POST /api/auth/login`
14 | - `GET /api/auth/profile` (Bearer token)
15 | - `GET /api/faqs`
16 | - `GET /api/faqs/search?q=configure`
17 | - `GET /api/faqs/:id`
18 | - `POST /api/faqs` (Bearer token)
19 | - `PUT /api/faqs/:id` (Bearer token)
20 | - `DELETE /api/faqs/:id` (Bearer token)
21 | - `POST /api/ai/answer` (Bearer token)
22 | - `POST /api/ai/generate-faq` (Bearer token)
```

Repository Files Not Reproduced as Source Code

.gitignore	Repository configuration file is visible in the supplied project tree, but its exact content was not available in the local source archive. It is intentionally not guaranteed to be reproduced.
package-lock.json	Generated automatically by npm. It records resolved dependency versions and integrity hashes; it is not application source code and can be regenerated with npm install.